

How Data Refresh Works

This guide explains, in plain language, when the BlockPro app refreshes data, what it actually re-downloads when something has changed, and what to expect if you're already partway through an inspection when a refresh runs.

It's a companion to the [Login Flow](#) document.

The short version

- The app refreshes data on login, when you pull down to refresh, and when you open certain screens.
 - It uses a **"last modified"** stamp on each asset's checklist to avoid re-downloading anything that hasn't changed.
 - There is no automatic background sync — refreshes only happen in response to something you do.
 - **If you're partway through an inspection, finish and submit it before refreshing other screens.** Refreshing can replace the underlying checklist, and unsubmitted answers live only in memory until you tap Submit.
-

When the app refreshes data

On login

The first time you sign in (or the first time after logging out), the app does a full **initial sync**: it downloads your buildings, then the assets in each building, then the checklists for each asset. This is covered in detail in the Login Flow document.

Pull-to-refresh

You can pull down on:

- **The buildings list** — refreshes the buildings on your account.
- **A building's assets list** — refreshes the assets inside that building.

Pull-to-refresh only refreshes what's on that screen — it does **not** trigger a full sync of everything.

Opening a screen

Some screens quietly refresh in the background as soon as you open them:

- **Opening a building** — re-checks the assets in that building.
- **Opening an asset's inspection screen or detail screen** — re-checks that asset's checklist to see if it's changed.

These background refreshes are deliberately quick. If nothing has changed, you'll barely notice them.

What does NOT happen

- The app does **not** sync automatically in the background.
- The app does **not** poll the server on a timer.
- The app does **not** sync when you bring it back from the background.

Refreshes only happen because you logged in, pulled to refresh, or opened a screen.

What gets updated when "modified dates" differ

Every checklist on the server has a **"last modified"** stamp showing when it was last changed. The app uses these stamps to avoid downloading data it already has.

Where the "last modified" date comes from

There is **no separate API call** just to ask "has this changed?". The app doesn't poll a dedicated timestamp endpoint.

Instead, the **"checklist last modified"** date is included **inside the assets response**. When the app fetches the assets in a building, every asset in that response already comes stamped with its current `checklistLastModified` date. The app simply stores that date alongside the asset, and uses it on the next refresh to decide whether the checklist itself is worth re-downloading.

In short: the cost of finding out whether anything has changed is just the normal assets call the app was going to make anyway. The savings come on the **checklist** side — the app only pulls the full checklist for assets whose timestamp has actually moved.

Here's what that looks like per entity:

Buildings

Whenever a buildings refresh runs, the app re-fetches **all** of your buildings. There's no date check at this level — it's a small amount of data, so the app keeps it simple.

Assets

Whenever an assets refresh runs (for a particular building), the app re-fetches **all** of that building's assets. Each asset in that response comes back stamped with its current **"checklist last modified"** date. The app stores this date on the asset.

This is the single trip that lets the app work out what's changed — no extra probe request is needed.

Checklists — this is where the date check matters

For each asset the app just fetched, it compares:

- The **"checklist last modified"** date the server just reported (from the assets response), against
- The date the app saved the last time it downloaded that checklist.

Then:

- **Dates match** → the checklist is skipped. Nothing further is downloaded for that asset.
- **Dates differ** (or no date is stored yet) → the checklist is re-downloaded **for that asset only**.

This is why the first sync after login can take a moment, but later refreshes are usually fast — most checklists haven't changed, so most are skipped.

What "re-downloaded" actually means

When a checklist is re-downloaded for an asset, the app:

1. Removes the existing chapters and questions stored on the device for that asset.
2. Inserts the fresh chapters and questions returned by the server.
3. Refreshes the read-only "existing remedials" shown alongside questions for context.

In other words, the app doesn't try to merge — it replaces the asset's checklist outright with whatever the server currently says.

What happens if an inspection is already underway

This section is important. Please read it.

How an in-progress inspection is held

While you're answering questions in an inspection, your answers and any photos you've selected are held **in the app's memory**. They are not saved to the device as you go. They are only sent to the server — and counted as a recorded inspection — when you tap **Submit**.

There is no draft, no auto-save, and no resume.

What a refresh can do during an inspection

Refreshes can still happen while you're inspecting. For example:

- You pull to refresh on another screen.
- You navigate to a building or another asset and the screen quietly re-checks it.

If a refresh decides that the checklist for the asset you're inspecting has changed on the server, the app will replace the stored chapters and questions for that asset with the new ones — even while you have an inspection in progress against it.

Your answers themselves stay in memory. They will still be sent when you Submit. But:

- The answers are tied to the **question wording you originally saw**.
- If the wording or the set of questions has since changed on the server, what you submit may not line up with the current checklist.

What this means for unsubmitted answers

Because in-progress answers are only in memory:

- If the app crashes before you submit, **your unsubmitted answers are lost**.
- If the app is force-closed or killed by the operating system before you submit, **your unsubmitted answers are lost**.
- Closing the app or switching to another app for a long time can also discard them.

What we recommend

- **Finish and submit an inspection in one sitting where possible.**
- **Don't pull to refresh other screens while you're partway through an inspection.**
- **Avoid hopping between screens unnecessarily while inspecting** — opening certain screens triggers a background refresh that could touch the checklist you're working on.
- If you have to pause, be aware that resuming later isn't supported — your answers won't be there when you come back.

Quick reference

When	What gets refreshed
Initial sync (after login)	Buildings → assets → any new or changed checklists
Pull-to-refresh on the buildings list	Buildings only

Pull-to-refresh on a building	Assets in that building only
Opening a building	Assets in that building
Opening an asset's inspection or detail screen	That asset's checklist — but only if its "last modified" date has changed
App resume / coming back from background	Nothing automatic

Why the "last modified" check exists

The check is there to keep refreshes fast and to keep mobile data use down. Without it, every refresh would re-download every checklist in full, which would be slow on a large account and wasteful when most checklists haven't changed.

Because the timestamp ships **inside the assets response**, the app doesn't pay for an extra "what changed?" round-trip — it gets the answer as a side-effect of the assets call it was making anyway.

In practice this means:

- A short, snappy sync after login is normal — most checklists are being skipped.
- A longer sync usually means several checklists have genuinely been updated on the server since you last synced.
- The very first sync after a new install (or after logging out and back in) is always the longest, because nothing is cached.